

and ending with the closing tag `</b>` inside which you can have PHP statements separated by semi-colons. The line of PHP code `<?php echo '<b> Hello World </b>';` passes the output `<b> Hello World </b>` to the body of the HTML script. Now, a Web browser will process this further by interpreting the HTML tag `<b>` which specifies bold text. This is why bold text is displayed on the Web browser as shown in Figure 2.

Regular expressions in PHP

Now that we know how to set up a server and run PHP scripts, it is time for us to discuss regular expressions in PHP. There are three sets of regular expression functions in PHP to choose from. These are the *preg* functions, *mb_ereg* functions and *ereg* functions. Out of these three, we will be discussing just one set of functions used for regular expression processing, the *preg* functions.

There are some good reasons to choose *preg* functions over the other two. First of all, *preg* is PCRE based. We have already discussed PCRE (Perl Compatible Regular Expressions) regular expression style in detail in the first two articles in this series. Those articles covered Python and Perl, both of which use PCRE style regular expressions. So, it is wise to use this style because then it is not necessary to discuss the syntax of the regular expressions used in PHP. All you have to do is just refresh the syntax you have learned while learning regular expressions in Python and Perl. This is one point in favour of *preg* functions, while there are some faults with the other two sets of regular expressions.

The *mb_ereg* functions are more complicated and are useful only if we are processing multi-byte character sets. We will come across multi-byte character sets only when processing languages like Korean, Japanese or Chinese that have a huge number of characters. As an aside, let me add, unlike most other languages which use an alphabet with a fixed number of characters, these languages have tens of thousands of logograms to represent different words.

Now, let us come back to our business; it would be unnecessary to burden learners by discussing the *mb_ereg* set of functions with no real benefit in sight. And what disqualifies the *ereg* set of functions? Well, they are the oldest set of functions but they were officially deprecated from PHP 5.3.0 onwards. Since we have decided to stick with the *preg* set of functions in PHP to handle regular expressions, we don't need any further discussion regarding the syntax, because we are already familiar with the PCRE syntax.

The main functions offered by the *preg* regular expression engine include *preg_match()*, *preg_match_all()*, *preg_replace()*, *preg_replace_all()*, *preg_split()*, and *preg_quote()*. The function *preg_match()* can give different results based on the number of parameters used in it. In its simplest form, the function can be used with just two parameters as *preg_match(\$pat, \$str)*. Here, the regular

expression pattern is stored in the variable *\$pat* and the string to be searched is stored in the variable *\$str*. This function returns true if the given pattern is present in the string and returns false if no match is found.

A simple PHP script using regular expressions

Now that we have some idea about the regular expression syntax and the working of one function in the *preg* set of functions, let us consider the simple PHP script called *regex1.php* shown below:

```
<html>
<body>
<?php
    $pat = '/You/';
    $str = 'Open Source For You';
    if(preg_match($pat,$str))
    {
        echo '<b> Match Found </b>';
    }
    else
    {
        echo 'No Match Found';
    }
?>
</body>
</html>
```

To view the output of this script, open a Web browser and type *localhost/regex1.php* on the address bar. The message 'Match Found' will be displayed on the Web browser in bold text. This script also tells us how the function *preg_match()* searches for a match. The function searches the entire string to find a match. Let us analyse the script *regex1.php* line by line. The HTML part of the code is straightforward and doesn't need any explanation. In the PHP part of the script, we have used two variables *\$pat* and *\$str*. The pattern to be matched is stored in the variable *\$pat* by the line of code *\$pat = '/You/'*. Here we are going for a direct match for the word 'You'. As you might have observed, the delimiters of the regular expression are a pair of forward slashes (/). The variable *\$str* contains the string which is searched for a possible match, and this is done by the line of code *\$str = 'Open Source For You'*. The next few lines of code have an *if-else* block to print some messages depending on the condition of the *if* statement.

In the line of code *'if(preg_match(\$pat,\$str))'* the function *preg_match()* returns true if there is a match and returns false if there is no match. In case of a match, the line of code *'echo ' Match Found ';'* inside the *if* block will print the message 'Match Found' in bold text. In case there is no match, the line of code *'echo 'No Match Found';'* in the *else* block will print the message 'No Match Found'.